LEARNER GUIDE

Blockchain Invoice Verification

with OpenClaw AI Agent Platform

Course Code	TGS-2022015374
Course Title	WSQ Business Transformation with OpenClaw
Duration	1 Day (7½ hours) · 09:30 – 17:00
Institution	Tertiary Infotech Academy Pte Ltd (UEN: 20120096W)
Labs	Labs 1 – 10 · Including Blockchain Invoice Verification

Learning Outcomes

By the end of this course, you will be able to:

LO1 Deploy OpenClaw on a Hostinger VPS or Docker Desktop local machine

LO2 Connect LLM providers — Groq (free), OpenAI OAuth, Ollama, or Default config

LO3 Set up Telegram (BotFather) and WhatsApp (QR pairing) messaging channels

LO4 Integrate productivity tools — AgentMail, Agent Browser, Firecrawl

LO5 Install and invoke skills from the ClawHub marketplace

LO6 Schedule automated tasks with cron jobs and monitor uptime with heartbeat

LO7 Apply security hardening — allowlists, auth tokens, audit logs

LO8 Use the OpenClaw dashboard for monitoring and gateway management

LO9 Select cost-effective LLM models and manage API spending

LO10 Build a Blockchain Invoice Verification system using SHA-256 and Flask

Lesson Plan

1 Day · 09:30 – 17:00 · 7½ hours

Time	Duration	Activity
09:30 – 09:50		Welcome, Digital Attendance & Overview
09:50 – 10:20		Topic 1: Overview of OpenClaw [30 min]
10:20 – 10:40		Topic 2: OpenClaw Applications + Blockchain Demo [20 min]
10:40 – 10:50		Break [10 min]
10:50 – 13:00		Topic 3: OpenClaw Setup and Configurations — Labs 1–5 [2 hrs 10 min]
13:00 – 13:30		Lunch Break [30 min]
13:30 – 16:00		Topic 3 (cont.) — Labs 6–10 incl. Blockchain [2 hrs 30 min]
16:00 – 16:10		Break [10 min]
16:10 – 16:50		Final Assessment [40 min]
16:50 – 17:00		Wrap-Up & Closing

Assessment

Final Assessment: Written / MCQ assessment on the LMS. Open book — slides and this Learner Guide allowed. Must be attempted for SSG funding. Covers all 10 labs and OpenClaw concepts.

Funding: This course is eligible for SSG (SkillsFuture Singapore) funding. Attendance and assessment completion are both required for funding claims.

Hands-On Labs

Each lab builds on the previous one. Complete them in order. Use the Killercoda Ubuntu playground for terminal access.

Lab 1 **OpenClaw Hosting**
Deploy OpenClaw on Hostinger VPS or Docker Desktop

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

OpenClaw runs as a persistent daemon or Docker container. Choose Option A (VPS) for production or Option B (Docker) for local development.

Option A — Hostinger VPS

Step 1 SSH into your VPS

```
ssh root@YOUR_VPS_IP
# Replace YOUR_VPS_IP with your actual Hostinger VPS IP address
```

Step 2 Install OpenClaw

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Step 3 Install and start the daemon

```
openclaw daemon install
systemctl enable --now openclaw
openclaw gateway status   # should show: Gateway: running   Port: 18789
```

Option B — Docker Desktop

Step 1 Pull the OpenClaw image

```
docker pull openclaw/openclaw:latest
# This downloads ~500 MB. Wait for 'Status: Downloaded newer image'
```

Step 2 Run the container

```
docker run -d --name openclaw \
  -p 18789:18789 \
  -v openclaw-data:/root/.openclaw \
  openclaw/openclaw:latest
```

Step 3 Open OpenClaw in your browser

```
http://localhost:18789
# Open this URL in Chrome or Firefox on your laptop
```

✓ **Test it** — `openclaw gateway status`
Expected: *Gateway: running Port: 18789*

Lab 2 — OpenClaw Model

Connect an LLM provider: Groq, OpenAI, Ollama, or config file

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

OpenClaw supports multiple LLM backends. Groq is free and requires no credit card — recommended for this lab.

Step 1 Option A — Groq (free tier, no credit card)

```
# Visit console.groq.com → sign up free → API Keys → Create key
openclaw model set groq --api-key YOUR_GROQ_KEY --model llama-3.1-70b-versatile
```

Step 2 Option B — OpenAI OAuth

```
openclaw model set openai --oauth   # copy the printed URL to your browser
# Browser: Sign in to OpenAI → click Allow
# Terminal: ✓ OpenAI OAuth token saved.
```

Step 3 Option C — Ollama (local, no API cost)

```
openclaw model set ollama --model llama3
```

Step 4 Option D — Edit config file directly

```
nano ~/.openclaw/openclaw.json
# Set: provider, model, apiKey
```

Step 5 Test the model connection

```
openclaw model test   # expect: Status: ✓ Connected
```

✓ **Test it** — `openclaw model test`
Expected: *Provider: groq Model: llama-3.1-70b-versatile Status: ✓ Connected*

Lab 3 — OpenClaw Channel

Set up Telegram (BotFather) and WhatsApp messaging channels

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

Channels let you interact with OpenClaw via messaging apps. Telegram is the easiest — WhatsApp requires a QR code scan.

Telegram

Step 1 Open BotFather in Telegram

```
# Open Telegram → search @BotFather → send /newbot
```

Step 2 Enter bot name and username, copy token

```
# Enter: Alfred Agent  (name)
# Enter: alfred_agent_bot  (username)
# Copy the API token shown
```

Step 3 Register and start channel

```
openclaw channel add telegram --token YOUR_BOT_TOKEN
openclaw channel start telegram
```

WhatsApp

Step 1 Scan QR code

```
# QR code appears in terminal
# WhatsApp → Linked Devices → Link a Device → scan
```

Step 2 Start channel

```
openclaw channel start whatsapp
```

✓ **Test it** — `openclaw channel list`
Expected: *telegram running | whatsapp running*

Lab 4 — OpenClaw Tools

Integrate AgentMail, Firecrawl, and Agent Browser

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

Tools extend OpenClaw's capabilities: AgentMail for email, Firecrawl for web scraping, Agent Browser for browsing.

Step 1 AgentMail — sign up and get API key

```
# Visit https://agentmail.to → create free account → copy API key
```

Step 2 AgentMail — add tool

```
openclaw tools add agentmail --api-key YOUR_AGENTMAIL_KEY
```

Step 3 Firecrawl — get fc-... key and add tool

```
# Visit https://www.firecrawl.dev → create account → copy fc-... key
openclaw tools add firecrawl --api-key YOUR_FC_KEY
```

Step 4 Agent Browser — get key and add tool

```
# Visit https://agent-browser.dev → get API key
openclaw tools add agent-browser --api-key YOUR_KEY
```

Step 5 Test tools in chat

```
# Send: 'Check my email'
# Send: 'Browse https://news.ycombinator.com'
# Send: 'Scrape https://openclaw.ai'
```

✓ **Test it** — `openclaw tools list`
Expected: *agentmail enabled | firecrawl enabled | agent-browser enabled*

Lab 5 — OpenClaw Skills

Install and invoke skills from the ClawHub marketplace

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

Skills are pre-built capabilities you can add to OpenClaw in one command. ClawHub is the marketplace for community and official skills.

Step 1 Browse ClawHub marketplace

```
# Visit https://clawhub.ai → explore available skills
```

Step 2 Install web-research skill (VPS / local)

```
npx clawhub@latest install web-research
```

Step 3 Install daily-briefing skill (Docker)

```
docker exec openclaw npx clawhub@latest install daily-briefing
```

Step 4 Test skill in Telegram chat

```
# Send: /web-research What is OpenClaw?
```

Step 5 Update a skill

```
npx clawhub@latest update web-research
```

✓ **Test it** — `openclaw skills list`
Expected: *web-research active | daily-briefing active*

Lab 6 — Cron Jobs and Heartbeat

Schedule automated tasks and monitor uptime

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

Cron jobs let OpenClaw run tasks on a schedule. Heartbeat sends regular pings to confirm the agent is alive.

Step 1 Create a daily morning summary cron

```
openclaw cron create --schedule '0 9 * * *' \
  --prompt 'Summarise my email' --channel telegram --name morning
```

Step 2 Create a weekly AI news cron

```
openclaw cron create --schedule '0 8 * * 1' \
  --prompt '/web-research Latest AI news' --channel telegram --name weekly
```

Step 3 List crons and run one immediately

```
openclaw cron list
openclaw cron run morning
```

Step 4 Enable heartbeat (30-minute interval)

```
openclaw heartbeat enable --interval 30m --channel telegram
```

Step 5 Check heartbeat status

```
openclaw heartbeat status
```

✓ **Test it** — `openclaw heartbeat status`
Expected: *Heartbeat: enabled Interval: 30m Channel: telegram*

Lab 7 — OpenClaw Security

Allowlists, auth tokens, audit logs, and hardening

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

Security hardening protects your OpenClaw instance from unauthorized access. Apply all steps before deploying to production.

Step 1 Store API keys as environment variables

```
export GROQ_API_KEY='gsk_...'
echo 'export GROQ_API_KEY=...' >> ~/.bashrc
```

Step 2 Set Telegram allowlist

```
openclaw channel config telegram --allowlist-add YOUR_TELEGRAM_USER_ID
```

Step 3 Set gateway authentication token

```
openclaw config set gateway.auth-token $(openssl rand -hex 32)
```

Step 4 Disable unused tools

```
openclaw tools disable exec
```

Step 5 Export and review audit logs

```
openclaw logs --last 50
openclaw logs --export ~/audit.json
```

✓ **Test it** — `openclaw config list | grep -E 'gateway|auth'`
Expected: *gateway.auth-token set | gateway.allowlist enabled*

Lab 8 — OpenClaw Dashboard

Monitor agent status, memory, and gateway via web UI

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

The dashboard provides a visual interface for monitoring memory files, managing the gateway, and reviewing chat history.

Step 1 Launch dashboard on local machine

```
openclaw dashboard   # opens http://localhost:18789
```

Step 2 VPS — create SSH port-forward tunnel

```
ssh -L 18789:localhost:18789 root@YOUR_VPS_IP   # keep terminal open
```

Step 3 Open dashboard in browser

```
# Open: http://localhost:18789 in a NEW browser tab on your LOCAL machine
```

Step 4 Explore Memory panel

```
# View MEMORY.md, SOUL.md, AGENTS.md, HEARTBEAT.md
```

Step 5 Restart gateway from dashboard

```
# Dashboard → Gateway → Restart → wait 5 seconds → refresh page
```

✓ **Test it** — `openclaw gateway status`
Expected: *Gateway: running Port: 18789 Dashboard: http://localhost:18789*

Lab 9 — OpenClaw Cost Saving

Switch models, compaction, and spending alerts

Practice: *https://killercoda.com/playgrounds/course/kubernetes-playgrounds/ubuntu*

API costs can add up quickly. Use free-tier models, Claude.ai subscription, or context compaction to keep costs near zero.

Step 1 Switch to cheapest Claude model (Haiku)

```
openclaw model set anthropic --model claude-haiku-4-5-20251001
```

Step 2 Use Claude.ai subscription (fixed monthly fee)

```
openclaw model set anthropic --subscription
```

Step 3 Use MiniMax free tier (zero API cost)

```
openclaw model set minimax --api-key KEY --model abab6.5s-chat
```

Step 4 Enable context compaction

```
openclaw config set context.compaction enabled
openclaw config set context.compaction-threshold 50000
```

Step 5 Set a monthly spending alert

```
openclaw usage stats
openclaw usage alert --threshold 5.00 --channel telegram
```

✓ **Test it** — `openclaw usage stats`
Expected: *Total spend this month: $0.00 | Compaction: enabled*

Lab 10 — Blockchain Invoice Verification

What is Blockchain? A blockchain is a chain of records (blocks) where each block contains data AND a SHA-256 hash of the previous block. If any record is altered, its hash changes and breaks the chain — making tampering immediately detectable. In this lab, each invoice is one block.

Use Case: Invoice Fraud Prevention

Finance teams receive supplier invoices that can be altered (amount, bank details, date) before payment is approved. OpenClaw's Blockchain Verification layer addresses this:

REGISTER — Supplier submits invoice → Flask API computes SHA-256 hash → stored in ledger
VERIFY — Finance re-submits invoice → hash recomputed → MATCH (authentic) or TAMPERED
FRAUD — Alert fired to Telegram channel instantly on tamper detection

SHA-256 Hashing Explained

SHA-256 is a one-way cryptographic function. The same input always produces the same 64-character hex digest, but changing even one character produces a completely different hash. This makes it ideal for tamper detection.

```
import hashlib, json

invoice = {"invoice_id": "INV-2026-001", "vendor": "Tertiary Infotech",
           "amount": 1500.00, "date": "2026-08-06"}

# Compute hash — canonical (sorted keys) so field order does not matter
data_str = json.dumps(invoice, sort_keys=True)
hash_val = hashlib.sha256(data_str.encode()).hexdigest()
print(hash_val)  # e.g.: a3f7c9d2b1e4...  (64 hex chars)

# Change amount to 9999.00 → completely different hash → tamper detected
```

API Endpoints

Endpoint	Description
`POST /register`	Submit invoice JSON; SHA-256 hash stored as blockchain proof
`POST /verify`	Re-hash invoice and compare to stored proof → MATCH or TAMPERED
`GET /invoices`	List all registered invoices with their proof entries
`GET /health`	Container health check — returns invoice count and status

Lab 10 — Blockchain Invoice Verification

Build and test the Flask blockchain REST API with Docker

Practice: *https://github.com/tertiarycourses/TGS-2022015374-OpenClaw/tree/main/labs/lab-10-blockchain-invoice*

Step 1 Clone the Lab 10 source from GitHub

```
git clone https://github.com/tertiarycourses/TGS-2022015374-OpenClaw.git
cd TGS-2022015374-OpenClaw/labs/lab-10-blockchain-invoice
```

Step 2 Build the Docker image

```
docker build -t openclaw/invoice-verify:latest .
```

Step 3 Run the blockchain verification container

```
docker run -d -p 5000:5000 --name invoice-verify \
  openclaw/invoice-verify:latest
curl http://localhost:5000/health
```

Step 4 Register an invoice

```
curl -X POST http://localhost:5000/register \
  -H 'Content-Type: application/json' \
  -d '{"invoice_id":"INV-2026-001","vendor":"Tertiary Infotech",
       "amount":1500.00,"date":"2026-08-06"}'
```

Step 5 Verify the invoice — same data (should MATCH)

```
curl -X POST http://localhost:5000/verify \
  -H 'Content-Type: application/json' \
```

```
  -d '{"invoice_id":"INV-2026-001","vendor":"Tertiary Infotech",
       "amount":1500.00,"date":"2026-08-06"}'
```

Step 6 Tamper test — change amount to 9999 (should TAMPERED)

```
curl -X POST http://localhost:5000/verify \
  -H 'Content-Type: application/json' \
  -d '{"invoice_id":"INV-2026-001","vendor":"Tertiary Infotech",
       "amount":9999.00,"date":"2026-08-06"}'
```

✓ **Test it** — `curl http://localhost:5000/health`
Expected: *{"invoices": 1, "status": "ok"}*

Expected Responses: Authentic → {"status": "VERIFIED", "match": true}
Tampered → {"status": "TAMPERED", "match": false} + Telegram alert fired

Assessment Preparation

Key concepts to revise before the MCQ assessment:

OpenClaw Architecture Gateway, Daemon, Memory files (MEMORY.md, SOUL.md, AGENTS.md, HEARTBEAT.md), Mission Control dashboard

LLM Providers Groq (free), OpenAI OAuth, Ollama (local), MiniMax (free tier), Claude.ai subscription

Channels Telegram BotFather setup, WhatsApp QR pairing, allowlist security

Tools AgentMail (email), Firecrawl (scraping), Agent Browser (browsing) — all require API keys

Skills ClawHub marketplace, install/update with npx clawhub@latest

Cron Syntax '0 9 * * *' = daily 9am, '0 8 * * 1' = weekly Monday 8am

Heartbeat Periodic ping to confirm agent is alive; --interval and --channel required

Security Auth token (openssl rand), allowlist, audit logs, disable unused tools

Cost Saving Context compaction threshold, Haiku model, usage alerts

Blockchain / SHA-256 Hash = fingerprint of invoice data. Same data → same hash. Any change → different hash = tamper detected. Chain: each block holds previous block's hash → breaking one breaks all.

Quick Reference — Key Commands

`openclaw gateway status`	Check gateway is running
`openclaw model test`	Verify LLM connection
`openclaw channel list`	List active channels
`openclaw tools list`	List enabled tools
`openclaw skills list`	List installed skills
`openclaw cron list`	Show scheduled cron jobs
`openclaw heartbeat status`	Check heartbeat
`openclaw logs --last 50`	View last 50 audit log entries
`openclaw usage stats`	Show API spending
`curl localhost:5000/health`	Lab 10: blockchain API health check